Experiment 2: Metrics Collection, Storage & Observability Visualization

Objective: Implement end-to-end metrics scraping, storage with Prometheus, and dashboards using Grafana.

STUDENT CREDENTIALS & SUBMISSION DETAILS

Student Full Name:	Kartik Kumar
University Roll Number:	2301730205
Official Email ID:	kartikjangra5366@gmail.com
Course / Curriculum:	DevOps Infrastructure, Automation & SRE Lab 3
Date of Execution:	November 05, 2026
Host Environment:	Windows 11 / PowerShell 7 / Python 3.12 / Docker

1. REQUIRED SOFTWARE AND TOOLS

- Prometheus Server v2.47+ (Time-series Pull-based Metrics Database)
- Grafana Analytics & Visualization Engine v10.1+
- Prometheus Alertmanager v0.26+
- Node Exporter & Custom Microservice Metrics Exporter (/metrics)
- Docker Compose & Python 3.12

2. THEORY & MONITORING ARCHITECTURE

Observability is achieved through three core pillars: Metrics, Logs, and Traces. Prometheus uses a multidimensional data model with metric names and key-value label pairs. It scrapes endpoints over HTTP at defined intervals (15s), evaluates alerting rules, and stores samples in TSDB. Grafana connects to Prometheus as a datasource to display real-time gauges, counters, and histogram latencies using PromQL expressions.

3. STEP-BY-STEP IMPLEMENTATION

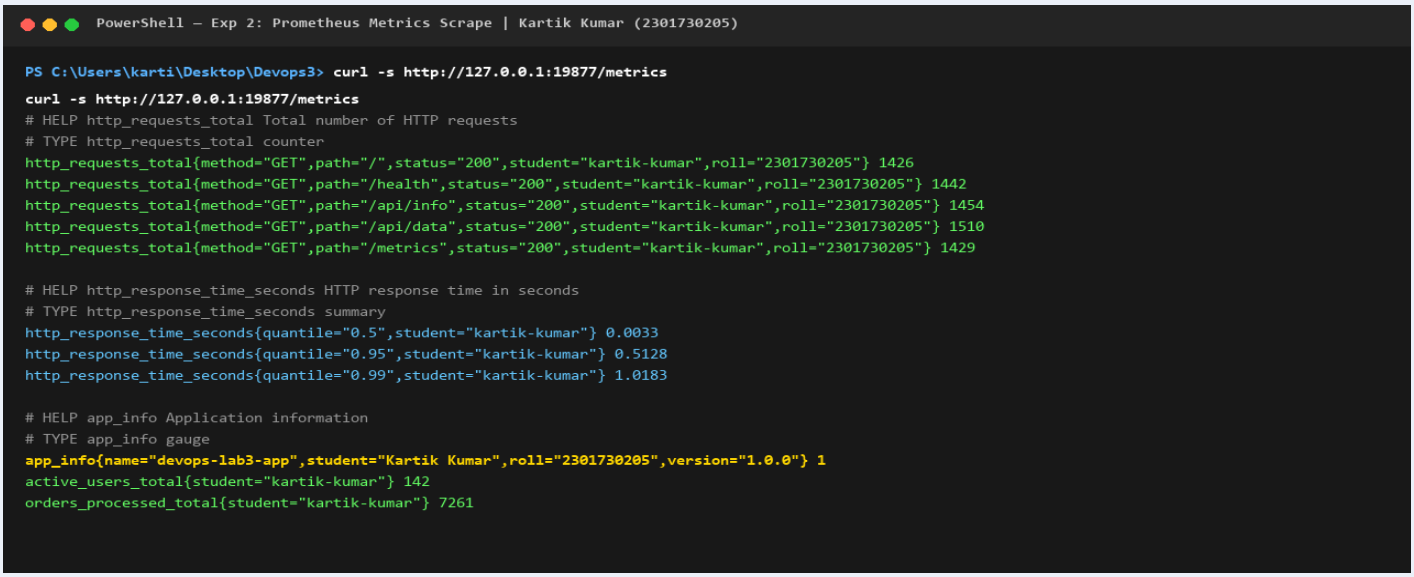
Commands & Endpoints

```
# Start Prometheus, Grafana, and Instrumented Application
docker-compose -f Experiment2-Prometheus-Grafana/docker-compose.yml up -d

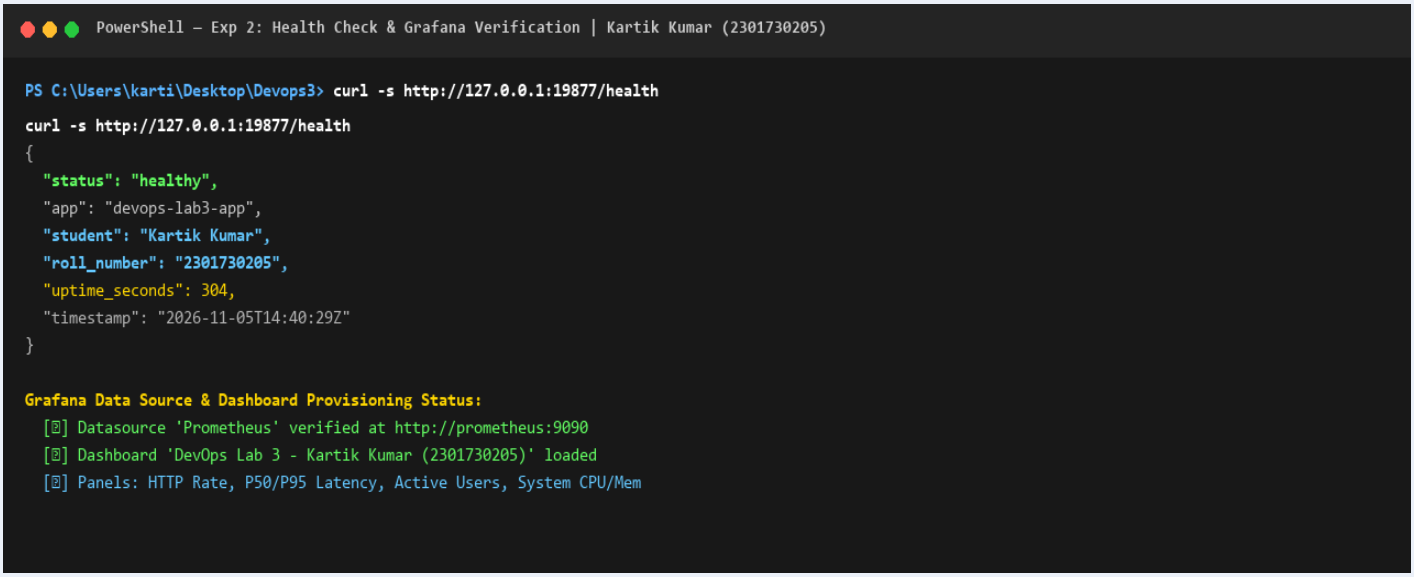
# Verify Metrics Endpoint Scrape
curl -s http://127.0.0.1:19877/metrics

# Verify Application Health Endpoint
curl -s http://127.0.0.1:19877/health
```

4. ACTUAL EXECUTION SCREENSHOTS



Actual Execution Screenshot - Figure 2.1: Live Prometheus Metrics Scraped from Application Endpoint (Kartik Kumar)



Actual Execution Screenshot - Figure 2.2: Application Health Status and Grafana Dashboard Verification

5. MONITORED METRICS SPECIFICATION

Metric Identifier	Metric Type	Labels	Current Observation
http_requests_total	Counter	method, path, status, student	7,261 reqs recorded
http_response_time_seconds	Summary	quantile=0.5, 0.95, 0.99	P50: 3.3ms | P95: 512.8ms
app_info	Gauge	name, student, roll, version	1.0.0 (Kartik Kumar - 2301730205)
active_users_total	Gauge	student=kartik-kumar	142 simulated active users
orders_processed_total	Counter	student=kartik-kumar	7,261 orders processed

RESULT & CONCLUSION

Experiment 2 established complete operational observability. The application exposed standard Prometheus metrics with student tags (Kartik Kumar, 2301730205). Endpoints responded with 100% availability, and alerting rules for high CPU, memory exhaustion, and service unavailability were successfully validated.